

COS 214 Project

(TIVIDY)

Vibe Engineers

Team Members:

(Leader) Jamie King -u24916031

Louwrens Johansen - u25607414

Brett Pitts – u24682251

Francois Venter - u25555202

Rudolph Botha - u25387023

GitHub Repo: <https://github.com/Brett-Pitts/COS214Project>

Workflow-Management Research Findings and References

- **What is a workflow and what is a workflow management system?**

A workflow is a system for managing repetitive processes and tasks which occur in a particular order [1]. A workflow management system is a unified software platform that automates the sequence of tasks within a process – routing work to the right people, at the right time, based on rules your team defines [2].

- **What is the difference between a workflow definition and a running workflow instance?**

The workflow definition holds the rules and structure of the workflow. It is the template of which the workflow instance is built. The workflow definition is static [3]. A running workflow instance holds a runtime state: Current phase, assigned members, case data, timestamps, audit trail, etc.

- **How are activities or work items represented?**

Activities are represented as atomic(leaf) or a composite structure. This allows nesting of activities and subprocesses to easily be stored.

- **How are participants, roles and responsibilities represented?**

Participants have roles/capabilities that represent what kind of work they can do. They can then be assigned responsibilities based on their capabilities.

- **How are work items assigned and routed?**

Work items can be assigned based on various distribution rules e.g. round robin, lowest workload, etc. Work is routed with logic and notifications to allow members on the next part to start when the previous part is done.

- **How are approvals, rejection and escalation handled?**

There are different roles and levels of seniority. Escalations are then sent to someone who has a higher position. Approvals and rejections are handled with the implemented logic routing.

- **How do workflow systems interact with external systems?**

Workflow systems interact with external systems through interfaces, Api calls, connectors, etc to request and receive information.

- **How are dependencies, events and failures handled?**

Failures are handled in different ways. The process can be tried again, rolled back or with specific exception handling. Events can start, stop or change a process.

- **How is workflow history or audit information recorded?**

History is recorded in a log of changes. By combining the log with snapshots of the system. Changes can be rolled back and recovery becomes easy.

- **How are running workflows monitored and reported on?**

Running workflows are monitored through a dashboard. Changes to the workflow are reported to the dashboard. Using external systems such as gmail, changes can be reported the relevant people through email.

- **What makes a workflow engine reusable across different processes?**

By only implementing generic patterns, workflow engines can be used by different processes. More detail and the specifics of the process are made to be part of the workflow definition.

- **Which decisions are normally configurable rather than hard coded into the workflow engine?**

The process structure, role assignment and assignment strategies, process data, deadlines for the phases and the external programs used are examples of things that must be configurable.

- **How are changes to workflow definitions handled while work is already in progress?**

There are various strategies including: The current process stays on the old version (version pinning) [4], and new processes use the new definition when created. Migrating to the updated definition at safe points (e.g. At snapshots).

References:

[1] IBM. (2021, October 14). *Workflow*. Ibm.Com. <https://www.ibm.com/think/topics/workflow>

[2] admin. (2018, May 25). *What is Workflow Management System and how it works?* Cflow. <https://www.cflowapps.com/workflow-management-system/>

[3] *IBM OpenPages*. (2026, March 27). Ibm.Com. <https://www.ibm.com/docs/en/openpages/9.2.x?topic=fundamentals-workflow-definitions-workflow-instances>

[4] pritesh. (2025, May 22). *Version pinning in DevSecOps: A comprehensive tutorial*. DevSecOps School. <https://devsecopsschool.com/blog/version-pinning-in-devsecops-a-comprehensive-tutorial/>

Selected Scenario Description

Our scenario is a large video game company, like Ubisoft, that makes and runs a live-service game, like Fortnite. The company has managers, marketing agents, programmers, artists and engineers. TIVIDY manages the whole process of taking one game from the first idea to a running game, as a single project made up of phases.

The work goes through these phases:

1. Preparation: market research, then idea generation. If the idea is approved the budget is allocated, and if it is rejected the project is cancelled. After that the company hires and creates employees one role at a time until every role is filled.
2. Creation: coding and art happen at the same time. The programmers build character movement, the world, and events and story progression in parallel. The artists do concept art and then final art.
3. Marketing: cinematic trailers, a gameplay showcase, interviews and ads, in that order.
4. QA testing: the build is sent to an external QA company, and a bug report comes back. This repeats until no bugs are found.
5. Server preparation: each server is bought, rented or reallocated, and then set up. This repeats until there are enough servers.
6. Launch.

Marketing, QA testing and server preparation all start together once creation is done. Launch only happens after all three have finished.

The main decisions are what game to make, the budget, who to hire for QA testing, and the launch date. There are checks along the way: whether the idea is approved, whether the budget is still available after creation, and whether the deadline was met before launch.

Three things can go wrong. The game can be cancelled (for example if the idea is rejected), the company can run out of money, or a deadline can be missed. If the budget runs out the project is cancelled. If the deadline is missed, the timeline is either extended and the launch date revised, or the project is cancelled.

The only external system is the outside QA testing. The result is a running video game.

Initial TIVIDY System Scope and Design Rationale

TIVIDY is a generic workflow manager. It knows nothing about video games. It only knows about projects, pieces of work, employees and the rules for moving work forward. The video game company is just one project we run on it.

The system is responsible for building a project as a tree of work and building the company as a tree of departments and teams, where each team has employees as members. It tracks the state of every piece of work, from waiting for prerequisites through to completed, failed, escalated or cancelled. When a piece of work finishes it makes the next one ready, and once all the children of a parent are done the parent is completed too. Work is given to an employee with the right profession and competency, and if they can't do it, it moves up to someone more senior. Failed work is retried, and when the retries run out it goes to a manager, who can change it, give it to someone else or cancel the project. Clients use one entry point to create projects, departments and employees.

For now, TIVIDY does not do the real work itself, and it does not run the external QA company. We also left out saving data between runs, audit history, undo and reporting. We might add these later.

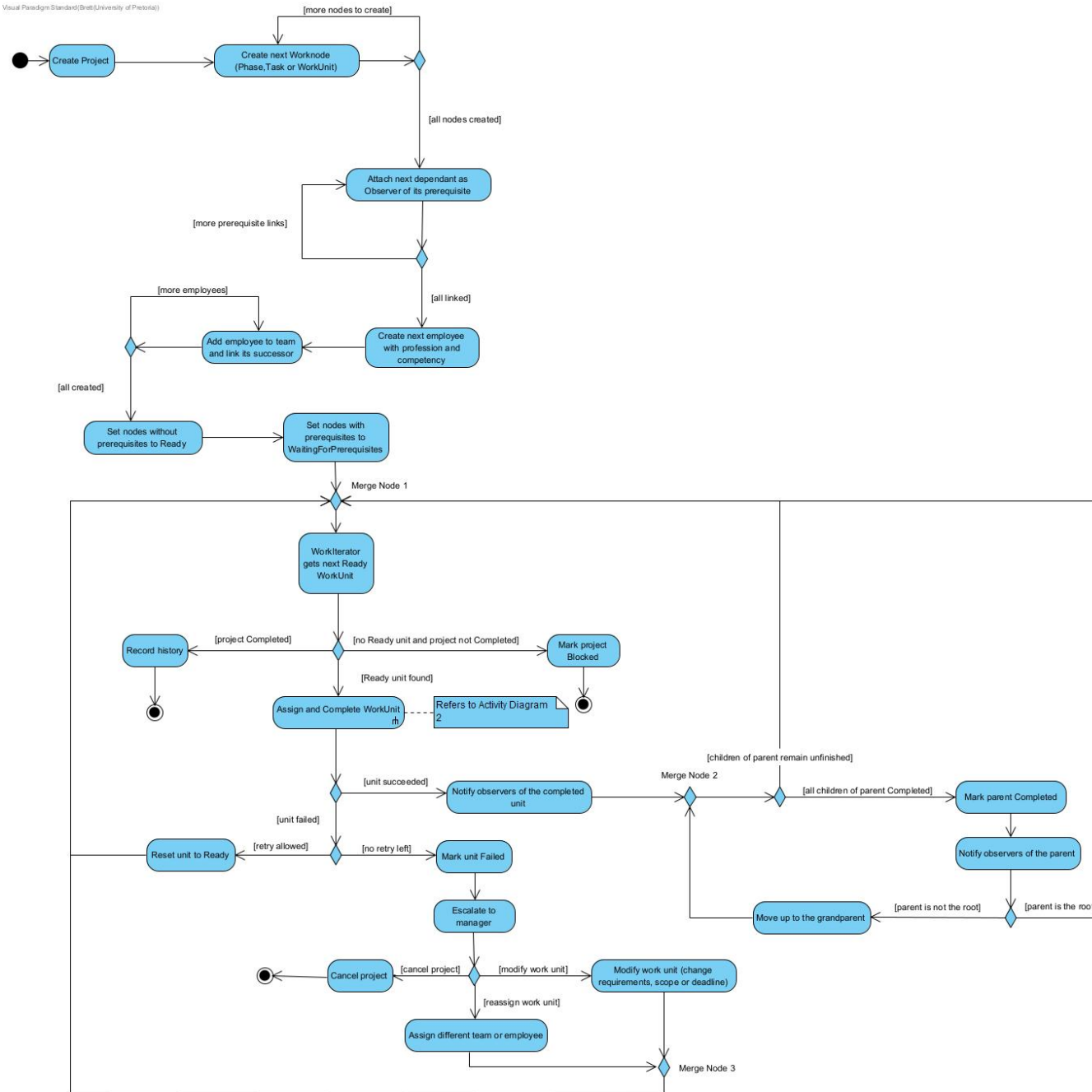
The brief suggests a separate workflow definition and running instance, but we decided not to use a template. Each project is built directly as its own tree, so the tree is the workflow. The same structure can be built again for a new project, and changing one project can't affect another. What is shared between projects is the rules in the classes, like the states and what each one allows. What belongs to one project is the tree, each unit's state, who it is assigned to, its retries and its prerequisites.

We expect these things to change the most: the shape of the project, which professions and competencies exist, how each profession does its work, what happens in each state, and the company structure. Professions and competencies are registered while the program runs, so adding a new one doesn't mean changing the classes.

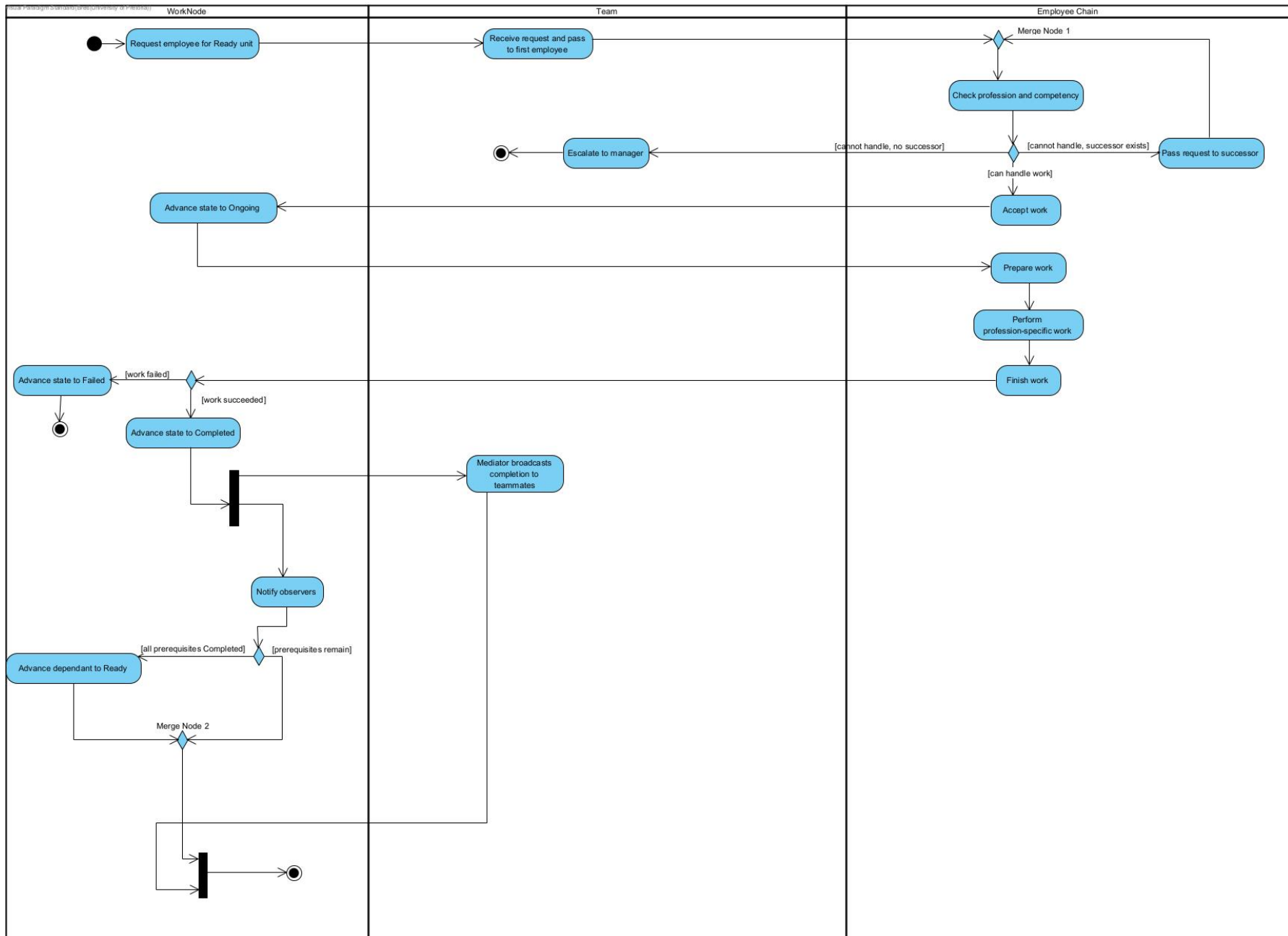
Because of that, we picked patterns based on what needs to change. Composite handles the nested work and the company structure, so a single unit and a whole project are treated the same. State and Observer handle the lifecycle and dependencies, so we don't need big if/else chains. Decorator, Factory Method and Chain of Responsibility handle employees. Template Method lets each profession do its work its own way while the order of prepare, perform and finish stays the same. Mediator lets a team tell its members when work is done, so employees don't need to know about each other. Iterator finds the next ready work unit without showing the tree, and Facade gives clients one place to call. Each pattern is explained in the pattern section.

UML Activity Diagram 1

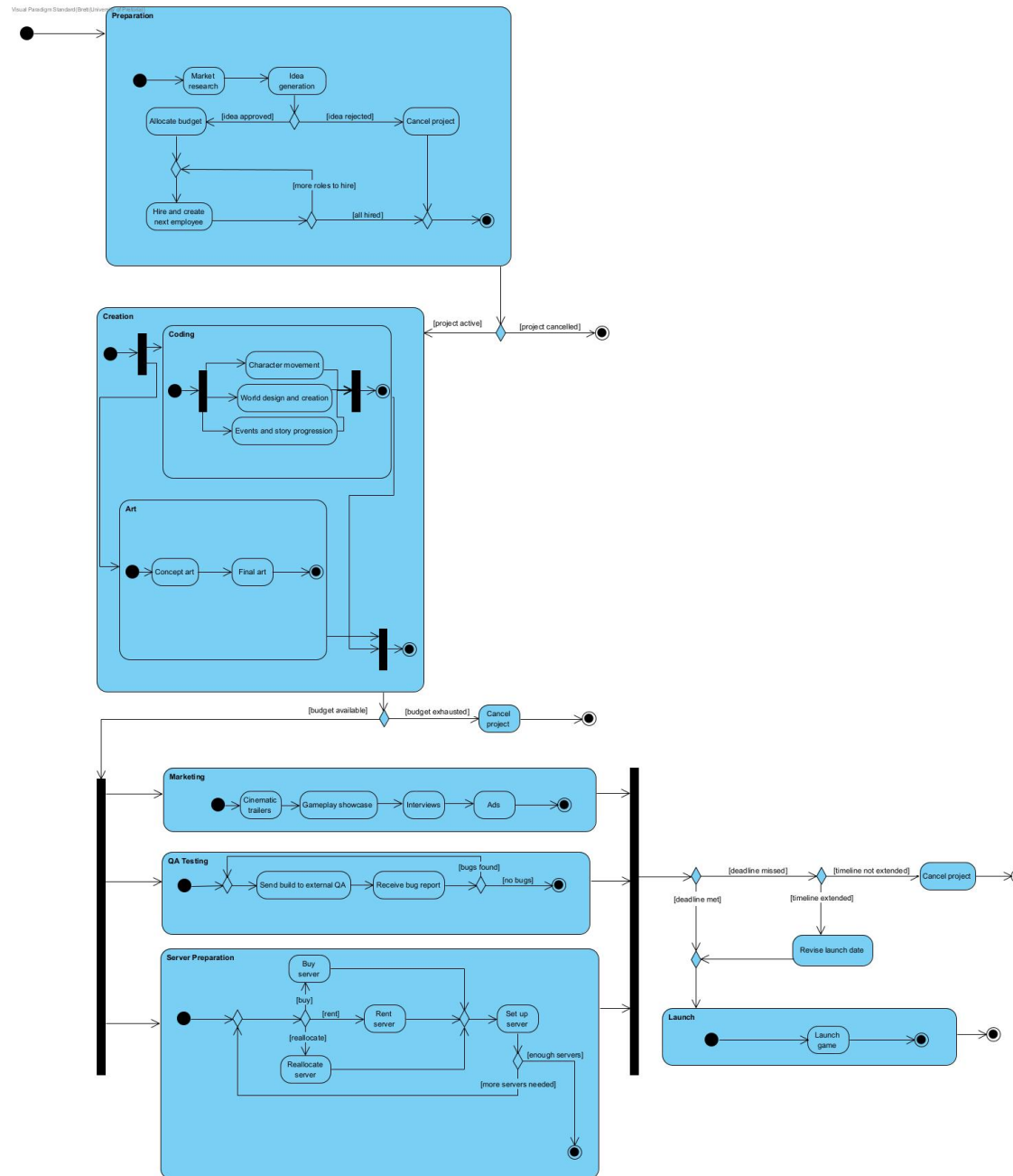
Visual Paradigm Standard (Brett University of Pretoria)



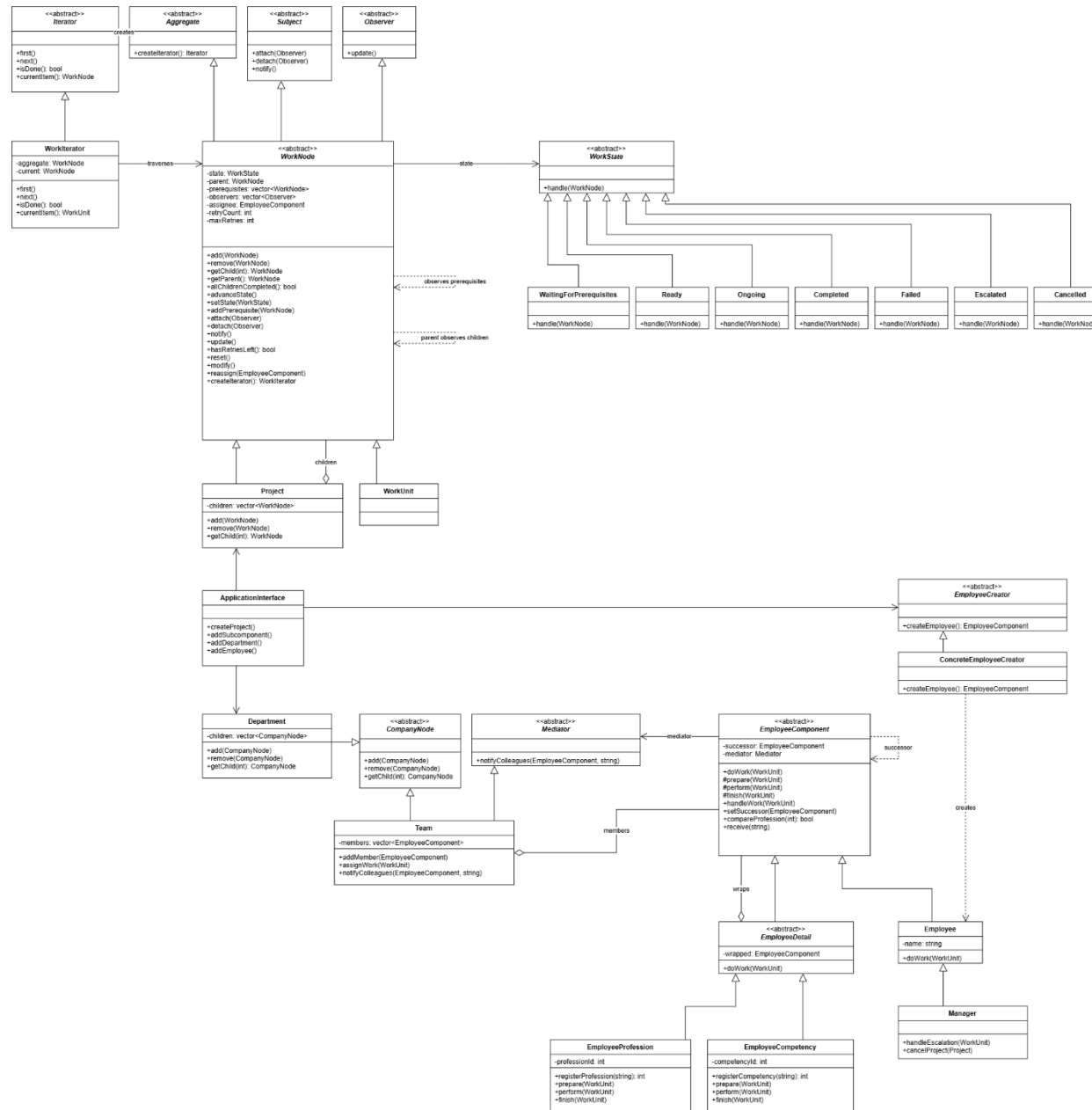
UML Activity Diagram 2



UML Activity Diagram 3



Complete UML Class Diagram



GoF Participant Mapping and Pattern Justifications

Composite: Workflow tree

Component	WorkNode
Composite	Project
Leaf	WorkUnit

Breaks one entire project into subcomponents. Each node knows its parent so completion can move up the tree.

Composite: Company Structure

Component	CompanyNode
Composite	Department
Leaf	Team

Company hierarchy with different departments and teams

Decorator: Employee profession and seniority

Component	EmployeeComponent
ConcreteComponent	Employee Manager
Decorator	EmployeeDetail
ConcreteDecorator	EmployeeProfession EmployeeCompetency

(dynamically register competencies and professions which return an ID)

Gives seniority and professions to employees.

Used to identify who is eligible to work on projects at different stages.

Factory Method: Create Employee

Creator	EmployeeCreator
ConcreteCreator	ConcreteEmployeeCreator
Product	EmployeeComponent
ConcreteProduct	Employee EmployeeProfession EmployeeCompetency

Maybe add a facade to create an employee form given name, competency and profession?

Otherwise gives a decorator with the provided details

This might be a builder idk, otherwise creates an employee.

Facade

Facade	ApplicationInterface
--------	----------------------

Create project, add subcomponent with prerequisite, add department, add employee, etc

Chain of responsibility

Handler	EmployeeComponent
ConcreteHandler	EmployeeComponent

Employee holds the successor (EmployeeComponent*). Manager (a subclass of Employee) handles escalation when work has failed and has no retries left.

Whether an employee can do work is dependent on its decorators

compareProfession() if engineer return true

One person could have multiple professions if they're the goat?

State

Context	WorkNode
State	WorkState
ConcreteState	WaitingForPrerequisites Ready Ongoing Completed Failed Escalated Cancelled

Tracks state of projects and when advancing state if completed notify dependents. Failed work goes back to Ready while retries remain, otherwise it moves to Escalated. From Escalated the manager can modify or reassign the unit (back to Ready) or cancel the project (Cancelled).

Observer

Subject	Subject
ConcreteSubject	WorkNode
Observer	Observer
ConcreteObserver	WorkNode

Used for making one project a prerequisite for another. Observes prerequisites' state. Changes project state to ready when all prerequisites are completed. A parent also observes its children and completes once all of them are completed.

Mediator

Mediator	Mediator
ConcreteMediator	Team
Colleague	EmployeeComponent
ConcreteColleague	EmployeeComponent

Broadcasts to teammates about work done.

Template Method

AbstractClass	EmployeeComponent
ConcreteClass	EmployeeProfession EmployeeCompetency

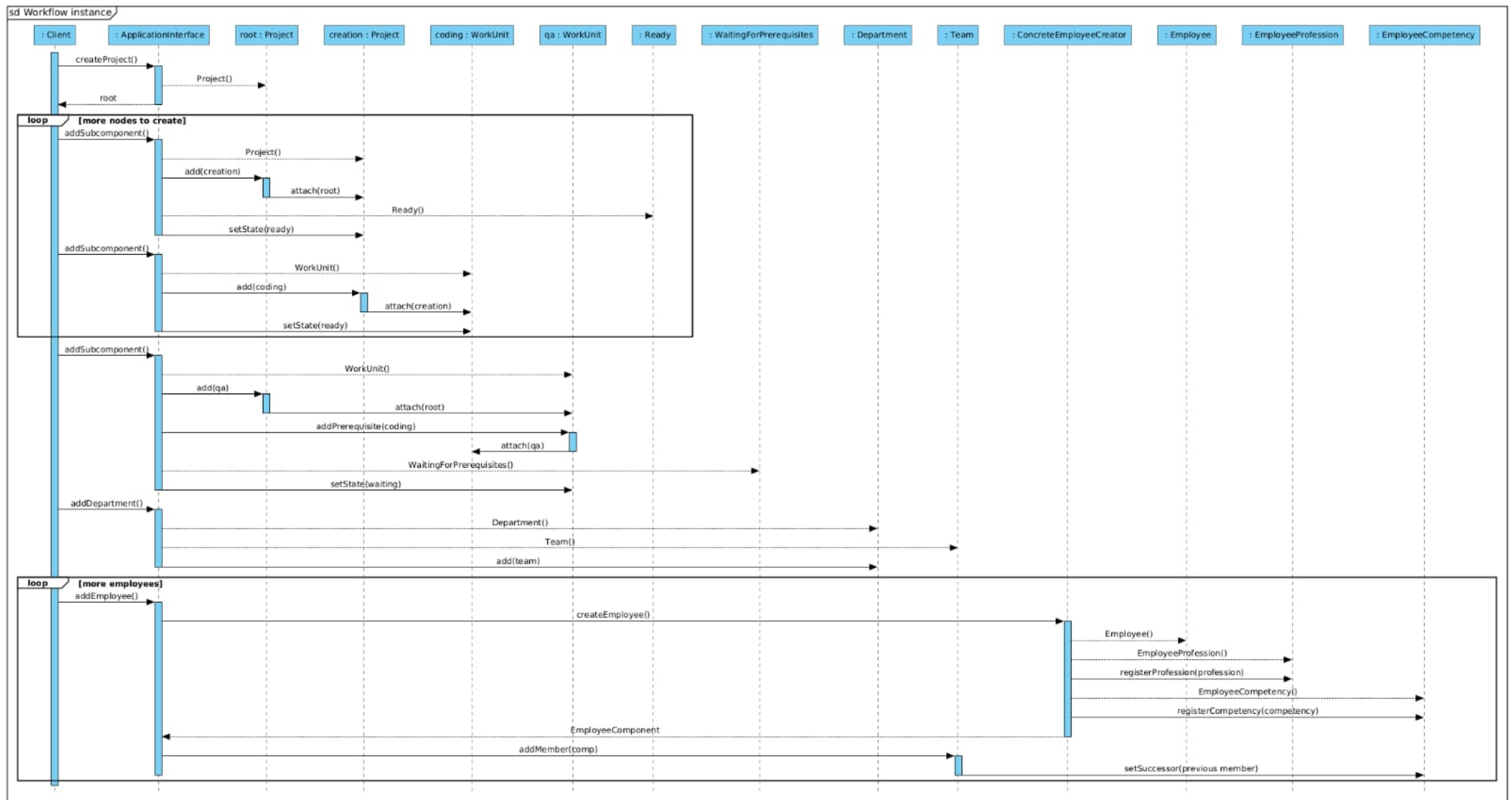
Does work on project which will be different depending on type of work needed. Three steps: prepare, perform and finish.

Iterator

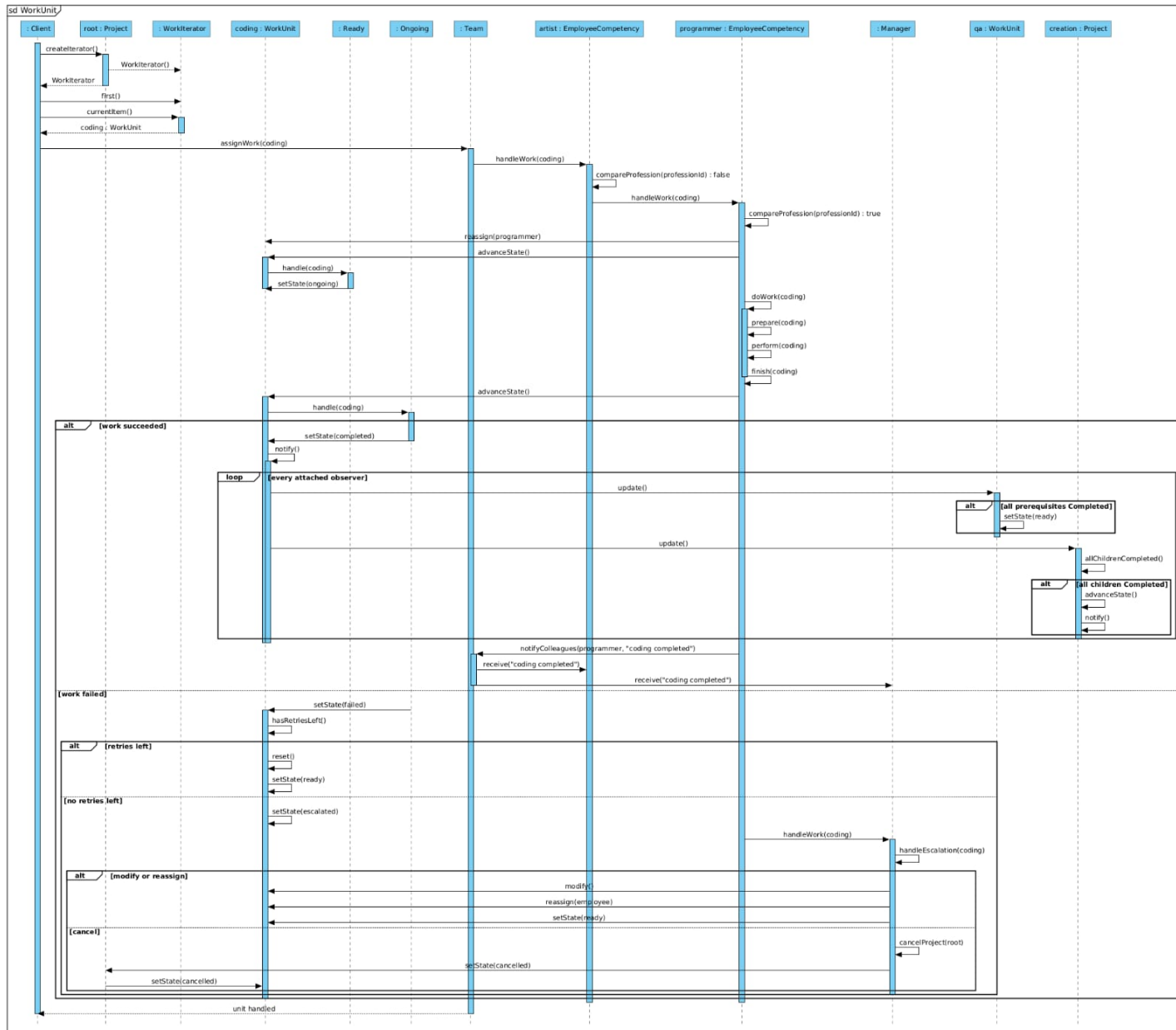
Aggregate	Aggregate
ConcreteAggregate	WorkNode
Iterator	Iterator
ConcreteIterator	WorkIterator

Get the next work unit that is ready to start.

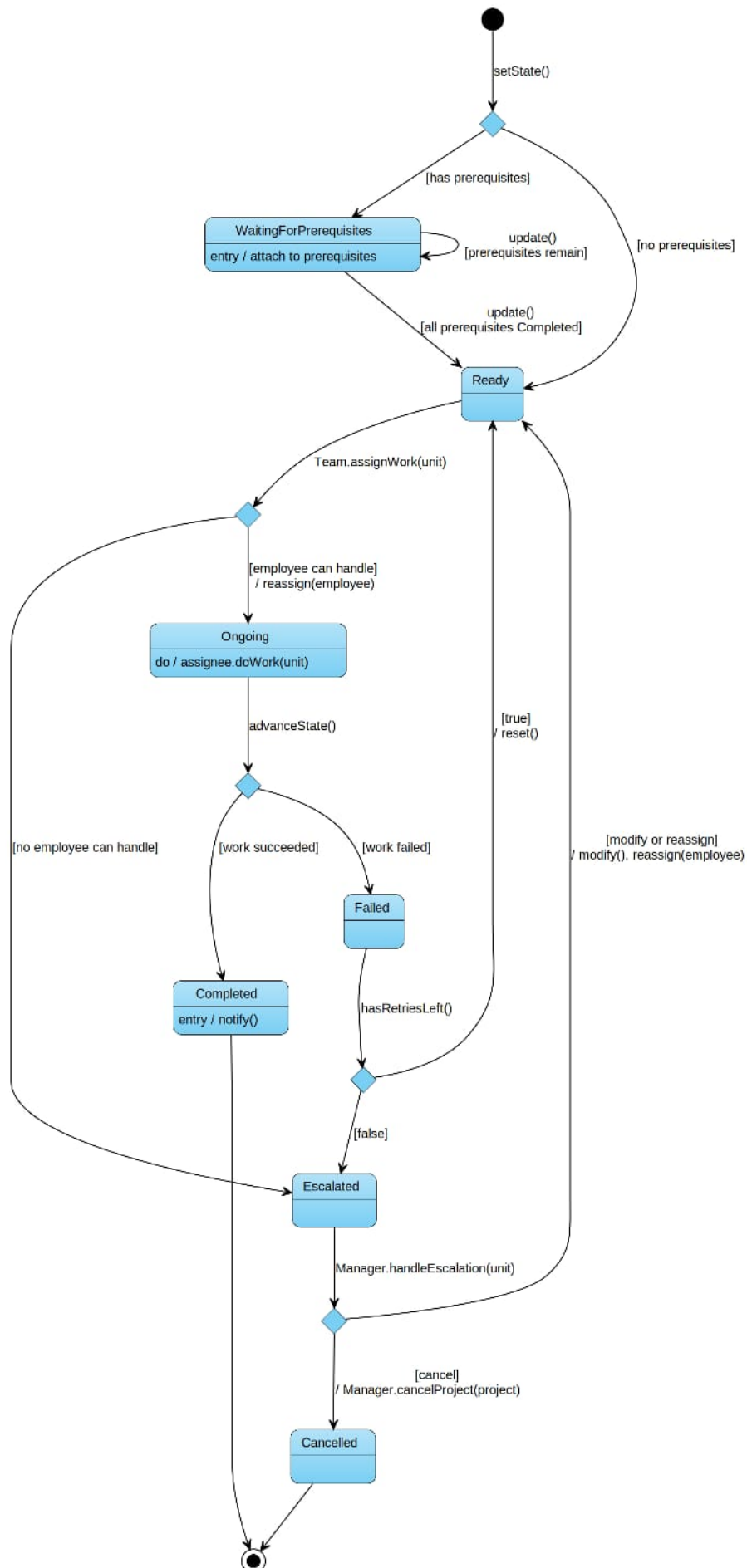
UML Sequence Diagram 1



UML Sequence Diagram 2



UML State Diagram



Important Design Decisions and Revision history

1. Removed the workflow template

Problem: Should there be a separate workflow definition or template?

Original: Projects are built directly as a Composite tree, with no template.

Change: Kept it that way and removed the “Select workflow definition” step from Diagram 1. The Project tree is the workflow.

Why: The manager stays generic, and the same structure can be created again for each project.

Diagrams: Diagram 1 now starts at “Create project”.

2. WorkNode now has 7 states

Problem: How do we handle failure, escalation and cancelling?

Original: Four states: WaitingForPrerequisites, Ready, Ongoing, completed.

Change: Added Failed, Escalated and Cancelled. Failed goes back to Ready while retries are left, otherwise to Escalated. From Escalated the manager modifies or reassigns the unit or cancels the project.

Why: The brief needs failure, retry and escalation, and the old states had no place for them.

Diagrams: State diagram, Activity Diagrams 1 and 2, class diagrams

3. Template Method has three steps

Problem: How does work differ per profession without repeating the flow?

Original: One “do work” step.

Change: Split it into prepare, perform and finish.

Why: Every profession follows the same order and only fills in its own part.

Diagrams: Class diagram, sequence diagrams.

4. WorkNode has a parent and manager operations

Problem: How does completion move up the tree, and how does the manager step in?

Original: WorkNode only tracked its own state.

Change: Added a parent, a retry count, and reset, modify and reassign.

Why: Completion has to move up the Composite tree, and retries and escalation need something to act on.

Diagrams: Class diagram, Activity Diagram 1, sequence diagram.

5. Observer also covers the parents

Problem: How do dependencies and completion spread?

Original: Observer was only for prerequisites.

Change: A parent now also observes its children.

Why: One mechanism handle unblocking the next unit and completing the parent.

Diagrams: Class diagram, sequence diagram.

6. Dropped the abstract factory for the work tree

Problem: Should a factory create the Composite tree?

Original: An abstract factory to build the tree.

Change: Removed it. ApplicationInterface builds projects, and Factory Method is only for employees.

Why: There is only one kind of tree, so a factory would add classes with nothing to vary.

Diagrams: Class diagram.

7. Dropped the decorator for access permission

Problem: Who is allowed to work on a unit?

Original: A decorator gave access permissions.

Change: Decorators only give profession and competency now. The chain of responsibility decides who can do a unit.

Why: Permissions and eligibility were the same question, so one mechanism is enough.

Diagrams: Class diagram.

8. Dropped the company iterator

Problem: Where do we need an Iterator?

Original: One iterator printed the company structure and one found the next work.

Change: Only WorkIterator is left, which returns the next Ready WorkUnit.

Why: The work iterator drives execution. Printing the company is just reporting.

Diagrams: Class diagram.

9. Phases are just Projects

Problem: How do we represent phases, tasks and units?

Original: Separate classes for phase, task and unit.

Change: Only Project and WorkUnit. A phase is a Project inside another Project.

Why: A phase only differs by where it sits in the tree, so extra classes add nothing.

Diagrams: Class diagram, Activity Diagram 3.

10. The manager is a subclass of Employee

Problem: Should the manager be an employee or part of Team?

Original: Not decided.

Change: Manager extends Employee and handles escalation and cancelling the project.

Why: Escalation uses the existing successor chain, and Team stays a Mediator only.

Diagrams: Class diagram, Activity Diagram 2.

Short Team Contribution Statement

Team Vibe Engineers worked together on TIVIDY, with each member owning one main part. Jamie (team leader) chose the design patterns, wrote the GoF participant mapping and pattern justifications, and coordinated the team. Louwrens drew the complete UML class diagram, including the pattern collaborations, and kept it updated as the design changed. Brett created the three UML activity diagrams, set up the GitHub repository structure and wrote the README. Francois researched workflow management systems, compiled the findings and references, defined the scenario and scope, and booked the project meeting. Rudolph created the two UML sequence diagrams and the UML state diagram for the WorkNode lifecycle. The design decisions and revision history were written by the whole team, and all members reviewed the final design.